# Übungsblatt 6: Multicore-Echtzeit und Shared Memory

**Musterlösung · Echtzeitsysteme (EZS)**

Prof. Dr.-Ing. Daniel Klünder

**Musterlösung**

## Zielsetzung

AMP auf dem STM32H755: Cortex-M7 Bare-Metal, Cortex-M4 mit FreeRTOS (CMSIS-RTOS2). Der Ping-Pong liegt in SRAM4. HSEM nur als Boot-Handshake, wie ihn Cube erzeugt. Im Kursrepository liegen keine `.ioc`-, HAL- oder Linker-Dateien. Board **NUCLEO-H755ZI-Q** nur verwenden, wenn das Projekt dieses Board ist. Takte, UART und HSEM-ID aus dem generierten Code abschreiben. Diese Anleitung baut hier nicht zu zwei ELF-Dateien.

---

## Aufgabe 1: Boot, nicht Reset-Halten

Beide Bootcodes können starten. Option Bytes können das ändern. Im üblichen Cube-Ablauf geht der M4 nach einer HSEM-Benachrichtigung in Stop. Der M7 setzt Takt und Shared-Block und gibt das HSEM frei. Stop ist kein gehaltenes Reset.

1. Generierten Handshake behalten. ID, HSEM-Takt und Wartebedingung notieren.
2. `HAL_PWREx_ReleaseCore` nicht ergänzen, wenn der Header sie nicht enthält.
3. Warten begrenzen. Bei Fehler einen Status ausgeben und stehen bleiben. Den Shared-Block nicht löschen, während der andere Kern schon läuft.
4. Pin Context ist Codegenerierung, keine Zugriffssperre. RCC nur einer Seite geben.

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Musterlösung Aufgabe 1

M7-Reihenfolge: auf M4-Stop warten, `SystemClock_Config` wie erzeugt, **dann** `ipc` initialisieren, **danach** HSEM freigeben. M4 löscht `ipc` nach dem Aufwachen nicht. Ein einzelner Core-Reset mitten im Ping-Pong ist nicht Teil dieser Basis. Ein gemeinsamer Reset läuft den Handshake neu.

---

### Aufgabe 2: Ein Block, zwei Images

`shared_data.h` enthält Typen, Konstanten und `extern`. Jedes Image hat genau eine Definitionsdatei.

```
#define IPC_MAGIC 0x49435031u
#define TURN_M7      1u
#define REQUEST_M4   2u
#define RESPONSE_M7  3u

typedef struct {
    uint32_t magic;
    uint32_t state;
    uint32_t payload;
} ipc_block_t;

extern volatile ipc_block_t ipc
    __attribute__((section(".shared")));

_Static_assert(sizeof(ipc_block_t) == 12, "layout");
_Static_assert(_Alignof(ipc_block_t) >= 4, "align");
```

Linker, nur wenn die Region schon `RAM_D3` = SRAM4 ist. Nicht ein zweites Mal anlegen. `.bss` beider Starts darf `.shared` nicht nullen.

```
.shared (NOLOAD) :
{
  . = ALIGN(32);
  KEEP(*(.shared))
  . = ALIGN(32);
} >RAM_D3
```

`NOLOAD` verhindert das Laden durch den Flash-Loader. Es befiehlt dem Startup nicht magisch, die Bytes nie anzufassen. Das prüfen Sie in den Startup-Tabellen.

Map-Soll, Schema bis die echten Maps da sind: dasselbe Symbol `ipc` in beiden Dateien, Adresse in `0x38000000`–`0x3800FFFF`, kein Heap darüber.

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

#### Musterlösung Aufgabe 2

M7, einmal, vor dem Release:

```
ipc.magic = IPC_MAGIC;
ipc.payload = 0;
ipc.state = TURN_M7;
__DMB();
ipc.state = REQUEST_M4;
```

Nach Reset und vor diesen Stores ist der Inhalt undefiniert. Zwei `uint8_t` plus `uint32_t` wären 8 Byte mit Payload ab Offset 4, nicht 12. Dieses Labor nutzt drei `uint32_t`.

---

### Aufgabe 3: Ping-Pong mit einem Schreiber

M4, FreeRTOS-Task. `osDelay(1)` wartet **einen Tick**. Die Tickfrequenz steht in `FreeRTOSConfig.h`.

```
void IpcTask(void *argument)
{
    (void)argument;
    for (;;) {
        if (ipc.magic != IPC_MAGIC || ipc.state != REQUEST_M4) {
            osDelay(1);
            continue;
        }
        __DMB();
        ipc.payload = ipc.payload + 1u;
        __DMB();
        ipc.state = RESPONSE_M7;
    }
}
```

M7 nach dem ersten Request. `HAL_Delay(500)` ist die sichtbare Pause, keine Latenzmessung. Timeout: drei Pausen ohne `RESPONSE_M7`, dann eine Zeile mit `state` und Ende. Nicht automatisch auf 0 setzen.

```
for (int n = 0; n < 3; n++) {
    uint32_t t0 = HAL_GetTick();
    while (ipc.state != RESPONSE_M7) {
        if ((uint32_t)(HAL_GetTick() - t0) > 2000u) { fail(); return; }
        HAL_Delay(500);
    }
    __DMB();
    log_u32(ipc.payload); /* 1, dann 3, dann 5 */
    ipc.payload = ipc.payload + 1u;
    __DMB();
    ipc.state = REQUEST_M4;
}
```

Runde 0: Payload 0 wird zu 1, Druck 1. M7 legt 2 hin. Runde 1: M4 macht 3. M7 legt 4 hin. Runde 2: M4 macht 5. Danach schreibt keiner, bis er den Zustand wieder besitzt.

`volatile` ersetzt weder `__DMB()` noch die MPU. Ein M4-Mutex gilt nicht für den M7.

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

**Musterlösung Aufgabe 3**

UART nur auf dem Kern, dessen Retargeting wirklich schreibt. Die ersten drei gedruckten Werte sind 1, 3 und 5. 0 ist der Startpayload und erscheint nicht als erste Antwortzeile. Während der M4-Task in `osDelay` liegt, können andere M4-Tasks laufen.

---

## Aufgabe 4: MPU und zwei Versuche

Basis, bevor der D-Cache an geht und bevor `ipc` benutzt wird: 64-KiB-Region ab `0x38000000`, Normal, TEX = 001, C = 0, B = 0, Execute Never, Schreiben erlaubt. Die Enum-Namen aus dem installierten `stm32h7xx_hal_cortex.h` übernehmen. Shareability so setzen, wie AN4838 sie für diesen Normal-Typ verlangt.

Versuch 2, separates Image, Neustart: D-Cache an, Region nachweislich cachebar und Write-Back, keine Clean/Invalidate-Paare. Erwartung: möglich ist ein veralteter Wert oder ein Timeout, nicht in jedem Build ein kompletter Stillstand. CCR und die MPU-Region lesen, bevor Sie den Fehler dem Cache zuschreiben.

Versuch 3 nur mit vollständig getrennten 32-Byte-Blöcken und einem Schreiber je Block. Ein Clean über eine Line, in der noch fremde neue Daten und eigene Dirty-Bytes liegen, ist kein Erfolg.

Zwischen den Varianten neu starten. Die MPU nicht umschalten, während beide Kerne schreiben.

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

**Musterlösung Aufgabe 4**

| Symptom | zuerst prüfen |
|---|---|
| M4 bleibt in Stop | HSEM-ID, nicht die MPU |
| Adresse verschieden | beide Map-Dateien |
| Magic nicht `IPC_MAGIC` | Init-Reihenfolge, `.bss` |
| M7 sieht 0 statt 1 | CCR und Region cachebar? |
| Timeout bei richtigem Zustand | Barriere oder falscher Kern schreibt |
| Debugger „steht“ | Halt-all-cores am Handshake |

Keine IPC-Latenz aus den 500 ms ablesen. Eine optionale Messung nimmt Pause und UART aus dem Fenster und nutzt GPIO-Marken. Zykluszähler der beiden Kerne nicht ohne gemeinsamen Takt subtrahieren. Solange kein Boardlauf protokolliert ist, bleiben die Zellen leer. Ein Compilerlauf wäre noch kein Hardwaretest; hier fehlt sogar das Cube-Projekt zum Bauen.